Factory Design Pattern

1. Classification, Intent & Exam Triggers

- **Category:** Creational Design Pattern (centralizes and abstracts object instantiation so client code does not call 'new ConcreteClass()' directly).
- **Intent (GoF Definition):** Defines an interface (or abstract method) for creating an object, but lets subclasses (Concrete Creators) decide which concrete class to instantiate.
- **Core Design Mantra:** Different Objects + Same Actions = Delegate object creation to a Factory! Instead of writing 'MySQLDatabase db = new MySQLDatabase()', program to the abstraction ('Database db = factory.createDatabase()') so the client never calls 'new' directly.
- **Primary Problem Solved (Tightly Coupled 'new' & if-else Code Smells):** Eliminates direct 'new ConcreteClass()' calls and sprawling if-else / switch creation blocks inside client code, which cause: (1) Tight Coupling (client depends on every concrete class like Pizza, Burger, MySQL, PostgreSQL), (2) Violation of Open/Closed Principle (adding Sushi or MongoDB forces modifying existing tested client code), (3) Repeated Object Creation (duplicated if-else creation scattered across multiple clients), and (4) Difficult Unit Testing (cannot easily replace or mock concrete objects).
- **Core OOP Principles:** Applies Abstraction & Interfaces (Product interface defines common operations), Polymorphism ('Vehicle v = new Car()' — reference type is the interface/superclass, actual behavior is decided at runtime by the concrete object), Inheritance & Method Overriding (ConcreteCreators override 'factoryMethod()'), and Open/Closed Principle (open for extension with new product/creator classes, closed for modification of client code).
- **How to Spot in Exam Questions :** Look for trigger phrases such as *"create different types / categories / variations of objects depending on user request or input," "centralize object creation without using* `new` *directly in client code," "replace long* `if-else` *or* `switch` *object-instantiation blocks,"* or *"support adding new product types in the future without modifying existing client code."* Typical question stem: *'An application needs to instantiate different categories of a common entity (where all types share the same actions/interface but have different concrete implementations) dynamically at runtime based on user selection, without tightly coupling the client to concrete classes. Which pattern should be used?'*

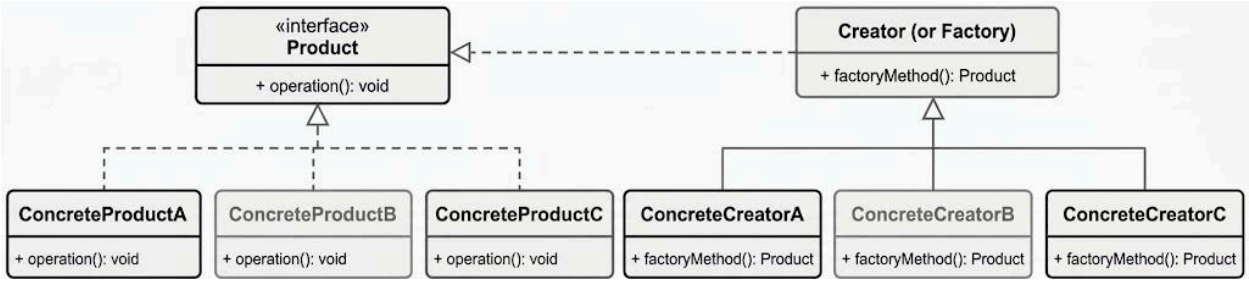
2. Pattern Structure, Participants & UML Relationships

Participant Role	Responsibility in Pattern
1. Product (Interface / Abstract Class)	Defines the common interface (methods/contract) for all objects the factory creates so the client can use every created object uniformly via polymorphism.
2. ConcreteProduct	Implements the Product interface (or extends abstract Product)

(Implementation Classes)	to provide specific behavior for a concrete type.
3. Creator (Factory) (Abstract Class / Interface)	Declares the abstract factory method (e.g., createPayment() / createKnife()) that returns a Product type. May also contain common workflow logic (e.g., orderKnife() calling sharpen(), polish(), package()).
4. ConcreteCreator (Concrete Factory Subclasses)	Extends/implements Creator and overrides the factory method to instantiate and return a specific ConcreteProduct (e.g., return new CreditCard()).
5. Simple Factory Variant (Single Helper Class)	A single concrete Factory class (e.g., BookFactory, KnifeFactory) with a creation method taking a 'type' parameter (if-else/switch) to return a Product. Frequently asked in 5-mark coding questions!

Four UML Relationships in Standard GoF Factory Method

- **1. Realization / Implementation (dashed line, hollow triangle: ConcreteProduct -> Product):** MySQLDatabase, PostgreSQLDatabase, and MongoDBDatabase implement the «interface» Database (or CreditCard, PayPal, Crypto implement «interface» Payment).
- **2. Generalization / Inheritance (solid line, hollow triangle: ConcreteCreator -> Creator):** CreditCardFactory, PayPalFactory, and CryptoFactory extend the abstract PaymentFactory (or BudgetKnifeStore extends KnifeStore).
- **3. Association / Dependency (solid/dashed arrow: Creator -> Product):** The Creator class references the abstract Product interface as the return type of its factoryMethod() without depending on any concrete product class.
- **4. Instantiation Dependency (dashed arrow '<<creates>>': ConcreteCreator -> ConcreteProduct):** Each ConcreteCreator depends on and instantiates its corresponding ConcreteProduct (e.g., CreditCardFactory -> CreditCard).
- **Abstract Class Rule:** (1) Although abstract methods must be declared inside an abstract class, an abstract class is NOT required to have abstract methods. (2) You can NEVER instantiate an abstract class directly with 'new'; an object can only be created through a concrete child class that inherits from it.



3. Implementation Code Structure

KnifeStore with Common Prep Steps + Factory Method

```
// Creator (KnifeStore) combining shared preparation steps with abstract createKnife()
public abstract class KnifeStore {
  public Knife orderKnife(String knifeType) {
    Knife knife;
    // Deferred to subclass via Factory Method!
    knife = createKnife(knifeType);

    // Common preparation workflow shared for all knives
    knife.sharpen();
    knife.polish();
    knife.packageKnife();
    return knife;
  }
  // Factory Method: Subclasses decide which concrete Knife to instantiate
  protected abstract Knife createKnife(String knifeType);
}

// Concrete Creator (BudgetKnifeStore)
public class BudgetKnifeStore extends KnifeStore {
  @Override
  protected Knife createKnife(String knifeType) {
    if (knifeType.equalsIgnoreCase("steak")) {
      return new BudgetSteakKnife();
    } else if (knifeType.equalsIgnoreCase("chefs")) {
      return new BudgetChefsKnife();
    }
    return null;
  }
}
```

5. Key Comparisons, Trade-Offs & Real-World Challenges

Factory Method vs. Simple Factory vs. Abstract Factory / Template Method

Comparison Area	Simple Factory (Idiom)	Factory Method Pattern (GoF)
1. Structure	One single concrete factory class (e.g., SimpleFactory / BookFactory) with all creation logic in one method.	Factory inheritance hierarchy: an abstract Creator base class and multiple ConcreteCreator subclasses.
2. Creation Logic	Uses an if-else or switch statement on a parameter (e.g., if type.equals('CREDIT_CARD')) to	Uses inheritance and method overriding (@Override createPayment()); each subclass

	decide which object to create.	instantiates its own product.
3. Extensibility (OCP)	Harder to extend: adding a new product type requires modifying the factory class's if-else block (violates strict OCP).	Easy to extend (Strictly follows OCP): add a new product by creating a new ConcreteCreator subclass with zero changes to existing code.
4. Flexibility & Coupling	Less flexible: the single factory class is tightly coupled to every ConcreteProduct class.	Highly flexible & loosely coupled: depends on abstractions rather than a central conditional block.
5. Best Used For	Small, stable systems with a fixed number of product types (or concise 5-mark exam factory outlines like 2025 Q3.3).	Large, evolving systems expecting frequent new product types, frameworks, and strict Open/Closed compliance.

Advantages, Disadvantages & Usage Rules

- **Key Advantages:**

  (1) Easy Unit Testing: Unit testing is a developer-performed technique where individual modules/procedures are tested in isolation. Because clients depend on the Factory and Product interfaces rather than 'new ConcreteClass()', developers can easily inject mock/stub products or mock factories during unit testing.

  (2) Loose Coupling & Code Transparency: Separates object creation from business logic; client code only knows the abstract Product interface.

  (3) Open for Extension, Closed for Modification (OCP): New product types (e.g., Crypto, MongoDB, Sushi) can be added without altering existing client code.

  (4) Reduces Code Redundancy & Improves Readability/Usability: If multiple clients instantiate the same set of classes, centralizing creation in a factory eliminates duplicate if-else blocks and keeps client code clean.
- **When to Use:** When the exact type of object to create varies at runtime, when future product extensions are expected, when the client should not know concrete implementation details, or when multiple implementations of the same interface exist.
- **When NOT to Use :** When the application is small/simple with only 1-2 fixed classes that never change (where adding Creator/ConcreteCreator hierarchies introduces unnecessary complexity and extra classes).

Real-World Challenges, Past Paper Mitigations & Reflection Questions

Challenge / Reflection Question	How to Explain & Overcome in the Exam
1. Subclass / Class Explosion (More Classes & Complexity)	Challenge: In strict Factory Method, adding every new ConcreteProduct requires creating a parallel ConcreteCreator subclass, doubling the number of classes and increasing structural complexity. Mechanism to Overcome [2022 Q2.5]: Use a parameterized Simple/Registry Factory (Map<String, Supplier<Product>>) or combine Factory with configuration files / Reflection so new types don't require

	separate factory subclasses.
2. 'Code Duplication Can Occur in Concrete Products' — Do You Agree?	Answer: YES, if ConcreteProducts (e.g., ExecutiveEmployee, DailyWorker, TemporaryStaff) implement a pure interface and repeat common steps (such as calculating the 8% EPF contribution or sharpening/packaging a Knife). Mechanism to Overcome: Combine Factory Method with the Template Method Pattern! Move shared invariant steps (like 8% EPF calculation or Knife preparation) into an Abstract Base Class so ConcreteProducts only override the varying parts.
3. Why is using 'new' everywhere considered bad practice?	Answer: Calling 'new ConcreteClass()' scatters creation logic across the codebase and tightly couples the client to concrete classes. Any constructor change or new product type forces edits across every client file, violating OCP and blocking isolated unit testing.
4. Which SOLID principles does Factory Method support, and why depend only on interfaces?	Answer: Supports (1) Open/Closed Principle (OCP — add new products without modifying client code), (2) Dependency Inversion Principle (DIP — high-level client depends on the Product interface abstraction, not low-level concrete classes), and (3) Single Responsibility Principle (SRP — separates creation logic from business usage).